



Universidad Veracruzana

Validaciones y capa de seguridad

Programación segura

Dr. Juan Manuel Gutiérrez Méndez

Propósito de la sesión

Incorporar una capa de seguridad para centralizar validaciones reutilizables y aplicar una primera regla de validación al formulario de bienes.

En esta versión se implementa completa la validación del identificador del bien. Las demás validaciones serán construidas por los estudiantes.

Al finalizar, el estudiante deberá poder:

- explicar la función de una capa de seguridad;
- crear un módulo de validadores reutilizables;
- validar el identificador del bien;
- conectar validaciones externas con `ModelForm` ;
- distinguir validación estructural y validación de dominio;
- proponer validaciones adicionales para otros campos.

Ruta incremental del proyecto

Versión 1

Autenticación, login y acceso protegido.

Versión 2

Modelo Bien, formulario básico y persistencia.

Versión 3

Capa de seguridad y validaciones reutilizables.

La versión 3 no cambia la arquitectura base; agrega una capa transversal para mejorar la revisión, reutilización y mantenimiento de reglas de validación.

¿Qué es una capa de seguridad?

La capa de seguridad es un conjunto de módulos auxiliares donde se concentran reglas transversales relacionadas con validación, permisos, auditoría o protección de datos.

En esta práctica se usará únicamente para validadores.

```
seguridad/  
  __init__.py  
  validadores.py
```

La capa de seguridad no sustituye a Django Forms ni a Models. Los complementa con reglas reutilizables y revisables.

¿Por qué separar validadores?

Separar las validaciones evita que toda la lógica quede dentro del formulario.

Sin capa de seguridad	Con capa de seguridad
Validaciones mezcladas en <code>forms.py</code> .	Validaciones concentradas en <code>seguridad/validadores.py</code> .
Reglas difíciles de reutilizar.	Reglas reutilizables entre formularios.
Mayor acoplamiento.	Mejor separación de responsabilidades.
Dificulta pruebas unitarias.	Facilita probar reglas específicas.

El formulario decide cuándo validar; la capa de seguridad define cómo se valida una regla reutilizable.

Estructura del proyecto en la versión 3

```
proyecto_seguro/  
  manage.py  
  
  config/  
    settings.py  
    urls.py  
  
  apps/  
    bienes/  
      forms.py  
      models.py  
      urls.py  
      views.py  
      templates/  
        bienes/  
          lista.html  
          formulario.html  
  
  seguridad/  
    __init__.py  
    validadores.py  
  
  templates/  
    base.html  
    registration/  
      login.html
```

Paso 1: crear la capa de seguridad

Desde la carpeta donde está `manage.py` , crear el directorio:

```
mkdir seguridad
```

Crear el archivo de validadores:

```
# Linux / macOS  
touch seguridad/validadores.py
```

Paso 2: revisar que no sea una app Django

La carpeta `seguridad/` no debe agregarse a `INSTALLED_APPS` .

En `config/settings.py` debe permanecer algo como:

```
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
  
    "apps.bienes",  
]
```

``seguridad/`` será un paquete Python auxiliar, no una aplicación Django con modelos, vistas, rutas o plantillas.

Paso 3: regla del identificador del bien

Para esta práctica, el identificador del bien debe cumplir el formato:

b-XXXX-año

Segmento	Regla
b	Letra fija en minúscula.
XXXX	Cuatro dígitos.
año	Año de creación del registro.
Rango del año	Desde 2000 hasta el año actual.

Ejemplos válidos:

b-0001-2026
b-1234-2026

Paso 4: crear validador del identificador

Editar:

```
seguridad/validadores.py
```

Contenido:

```
import re
from datetime import date

from django.core.exceptions import ValidationError

PATRON_ID_BIEN = re.compile(r"^b-\d{4}-\d{4}$")

def validar_identificador_bien(valor):
    if not PATRON_ID_BIEN.match(valor):
        raise ValidationError(
            "El identificador debe tener el formato b-XXXX-año."
        )

    anio = int(valor.split("-")[2])
    anio_actual = date.today().year

    if anio < 2000 or anio > anio_actual:
        raise ValidationError(
            f"El año debe estar entre 2000 y {anio_actual}."
        )
```

Lectura del validador

Elemento	Función
<code>re.compile(...)</code>	Define la expresión regular.
<code>^</code>	Indica inicio de cadena.
<code>b-</code>	Exige que el identificador inicie con <code>b-</code> .
<code>\d{4}</code>	Exige cuatro dígitos.
<code>-\d{4}</code>	Exige guion y cuatro dígitos para el año.
<code>\$</code>	Indica fin de cadena.
<code>ValidationError</code>	Reporta el error al formulario de Django.

El validador no guarda datos. Solo acepta o rechaza el valor recibido.

Paso 5: conectar el validador al formulario

Editar:

```
apps/bienes/forms.py
```

Importar el validador:

```
from django import forms
from .models import Bien

from seguridad.validadores import validar_identificador_bien
```

Actualizar el formulario:

```
class BienForm(forms.ModelForm):  
    class Meta:  
        model = Bien  
        fields = [  
            "identificador",  
            "descripcion",  
            "marca",  
            "valor",  
            "estatus",  
        ]
```

Paso 6: validar el campo identificador

Completar `apps/bienes/forms.py` .

```
class BienForm(forms.ModelForm):
    class Meta:
        model = Bien
        fields = [
            "identificador",
            "descripcion",
            "marca",
            "valor",
            "estatus",
        ]

    def clean_identificador(self):
        identificador = self.cleaned_data["identificador"].strip()
        validar_identificador_bien(identificador)
        return identificador
```

El método `clean_identificador()` se ejecuta cuando Django valida ese campo del formulario.

Archivo completo forms.py

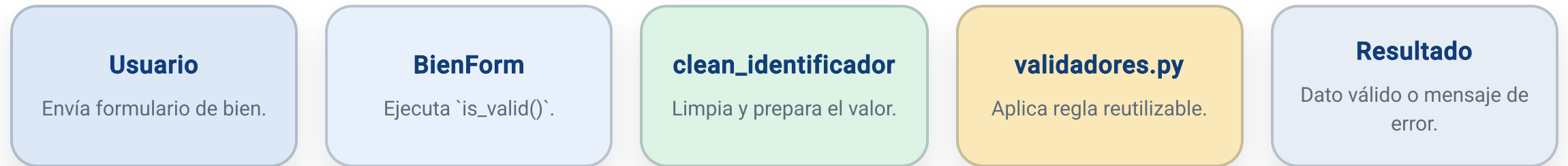
```
from django import forms
from .models import Bien

from seguridad.validadores import validar_identificador_bien

class BienForm(forms.ModelForm):
    class Meta:
        model = Bien
        fields = [
            "identificador",
            "descripcion",
            "marca",
            "valor",
            "estatus",
        ]

    def clean_identificador(self):
        identificador = self.cleaned_data["identificador"].strip()
        validar_identificador_bien(identificador)
        return identificador
```

Flujo de validación aplicado



El formulario conserva el control del flujo, pero delega la regla específica a la capa de seguridad.

Qué ocurre si el identificador es válido

Ejemplo:

```
b-0001-2026
```

Secuencia:

1. El usuario envía el formulario.
2. La vista construye `BienForm(request.POST)` .
3. Se ejecuta `form.is_valid()` .
4. Django llama `clean_identificador()` .
5. El validador acepta el formato y el año.
6. El registro se guarda con `form.save()` .

Un valor válido permite continuar el flujo hacia el modelo y la base de datos.

Qué ocurre si el identificador es inválido

Ejemplo:

```
bien-001
```

Secuencia:

1. El usuario envía el formulario.
2. Se ejecuta `form.is_valid()` .
3. El validador detecta formato incorrecto.
4. Se genera `ValidationError` .
5. Django asocia el error al campo `identificador` .
6. La plantilla muestra el mensaje al usuario.

Un valor inválido no se guarda. El formulario regresa al usuario con mensajes de error controlados.

Casos de prueba del identificador

Caso	Entrada	Resultado esperado
Formato válido	b-0001-2026	Aceptado.
Prefijo incorrecto	x-0001-2026	Rechazado.
Sin guiones	b00012026	Rechazado.
Menos dígitos	b-001-2026	Rechazado.
Año menor a 2000	b-0001-1999	Rechazado.
Año futuro	b-0001-2030	Rechazado si supera el año actual.

Validaciones que realizarán los estudiantes

Los estudiantes deberán completar validaciones para los demás campos.

Campo	Validación esperada	Responsable
descripcion	Longitud mínima, caracteres permitidos y texto significativo.	Estudiante
marca	Longitud mínima y caracteres permitidos.	Estudiante
valor	Numérico, positivo y dentro de rango razonable.	Estudiante
estatus	Valor permitido según opciones del modelo.	Modelo / estudiante
identificador	Formato b-XXXX-año y rango del año.	Docente / ejemplo base

La actividad consiste en extender el patrón de validación usado en el identificador hacia los demás campos.

Plantilla de trabajo para nuevas validaciones

En `seguridad/validadores.py`, los estudiantes pueden agregar funciones como:

```
def validar_descripcion(valor):  
    # 1. Limpiar espacios  
    # 2. Verificar longitud mínima  
    # 3. Rechazar caracteres no permitidos  
    # 4. Retornar valor limpio o lanzar ValidationError  
    pass  
  
def validar_marca(valor):  
    # Definir reglas y mensaje de error  
    pass  
  
def validar_valor(valor):  
    # Verificar que sea positivo y razonable  
    pass
```

Cada función debe tener una responsabilidad clara y mensajes de error comprensibles

Plantilla de conexión en forms.py

Los estudiantes deberán conectar sus validadores desde BienForm .

```
def clean_descripcion(self):  
    descripcion = self.cleaned_data["descripcion"]  
    # return validar_descripcion(descripcion)  
    return descripcion
```

```
def clean_marca(self):  
    marca = self.cleaned_data["marca"]  
    # return validar_marca(marca)  
    return marca
```

```
def clean_valor(self):  
    valor = self.cleaned_data["valor"]  
    # return validar_valor(valor)  
    return valor
```


El método `clean_()` debe devolver el valor limpio si es válido, o lanzar `ValidationError` si no cumple la regla.

Validación estructural vs validación de dominio

Tipo de validación	Ejemplo	Lugar frecuente
Estructural	<code>max_length=20</code>	<code>models.py</code>
Tipo de dato	<code>DecimalField</code>	<code>models.py</code> / <code>ModelForm</code>
Opción permitida	<code>choices</code>	<code>models.py</code>
Formato institucional	<code>b-XXXX-año</code>	<code>seguridad/validadores.py</code>
Regla contextual	Año entre 2000 y año actual	<code>seguridad/validadores.py</code>
Mensaje de error	Explicación para el usuario	<code>forms.py</code> / <code>validador</code>

La validación estructural define la forma básica del dato. La validación de dominio define reglas propias del problema.

Vista y plantilla no cambian

La vista `crear_bien` puede mantenerse igual.

Formulario HTML no cambia

La plantilla `formulario.html` también puede mantenerse igual.

Al cambiar la lógica del formulario, la vista conserva su responsabilidad de coordinar el flujo sin conocer cada regla de validación.

¿Por qué no cambia el template?

El template muestra los errores que el formulario genera.

Componente	Responsabilidad
<code>validadores.py</code>	Define reglas de validación.
<code>forms.py</code>	Ejecuta validaciones por campo.
<code>views.py</code>	Llama <code>form.is_valid()</code> .
<code>formulario.html</code>	Presenta campos y errores.

La plantilla no necesita conocer la expresión regular ni las reglas internas. Solo muestra los errores asociados al formulario.

Paso de verificación

Ejecutar:

```
python manage.py check
```

Levantar el servidor:

```
python manage.py runserver
```

Probar la ruta:

```
http://127.0.0.1:8000/bienes/nuevo/
```

Probar un identificador inválido:

```
bien-001
```

Resultado esperado:

```
El identificador debe tener el formato b-XXXX-año.
```

Actividad para estudiantes

Completar la capa de seguridad agregando validaciones para los campos restantes.

Campo	Criterios mínimos
descripcion	No vacía, longitud mínima, sin caracteres peligrosos.
marca	No vacía, longitud mínima, caracteres permitidos.
valor	Mayor que cero y dentro de rango razonable.
estatus	Debe corresponder a una opción válida.

Entregables:

- código de validadores;
- conexión en `BienForm` ;
- tabla de pruebas válidas e inválidas;
- captura de errores mostrados en el formulario.

Estructura final de la versión 3

```
proyecto_seguro/  
  manage.py  
  
  config/  
    settings.py  
    urls.py  
  
  apps/  
    bienes/  
      forms.py  
      models.py  
      urls.py  
      views.py  
      templates/  
        bienes/  
          lista.html  
          formulario.html  
  
  seguridad/  
    __init__.py  
    validadores.py  
  
  templates/  
    base.html  
    registration/  
      login.html
```


Errores comunes

Error	Consecuencia
Registrar <code>seguridad</code> en <code>INSTALLED_APPS</code>	Puede confundirse con una app Django.
Olvidar <code>__init__.py</code>	Python puede no reconocer la carpeta como paquete.
Importar desde <code>apps.seguridad</code>	Fallará si la carpeta está fuera de <code>apps/</code> .
No devolver el valor limpio	El formulario puede perder el dato validado.
Usar <code>ValidationError</code> incorrectamente	El error no se mostrará bien en el campo.
Validar solo en HTML	La petición puede manipularse desde el cliente.

Referencias de consulta

Recurso	Uso recomendado
Validación de formularios	Métodos <code>clean()</code> y <code>clean_<campo>()</code> .
ModelForm	Relación entre modelo y formulario.
ValidationError	Reporte controlado de errores.
Expresiones regulares en Python	Validación de formatos.
Seguridad en Django	Revisión de controles incorporados.

Gracias
¿Preguntas?